

WEEK 11 · THE DATA LAYER: JSON, LOCALSTORAGE, ASYNC, FETCH

IT IS STILL THERE TOMORROW, AND THE WORLD OUTSIDE IS SLOW

Last week the page got one source of truth. Today that truth survives a reload — and learns to ask somebody else's computer.

WHERE WE ARE

Week 11 of 12. The class library works, and forgets everything when you press F5.

THE NEXT 180 MINUTES

Five cycles, you typing in every one: the storage layer · data that is not data · fetch, then break it · five states, one field · a data layer from Claude. Break at 85, the assignment at 145.

WHAT THIS UNLOCKS

The project's two layers that talk to the world — `js/storage.js` and `js/api.js` — in the exact shape the week-13 exam reads.

Two new layers.
`render.js` learns about neither.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the pantry: saved on the disk, and a catalogue from the network reload open

המזווה הרשימה פריט סיכום

המזווה

שם

כמות

מדף

הוסף

חיפוש

מדף

מיון

לפי שם

הכול

מוצג: 0

נקה הכול



targets/week-11.png · 1280



375 · the same page, narrower

► **THE POINT** — add an item, reload: it is there. Delete the key: an **EMPTY** pantry, not the demo data. Search the catalogue with the network off: a sentence, and the local copy. You type every piece once in a cycle first, on the class library.

AI THIS WEEK

PARTNER. IT WRITES A DATA LAYER FAST — YOU CHECK THE THREE PLACES IT BREAKS.

LAYER · WEEKS 8–12 · PARTNER

ALLOWED

Claude writes `storage.js` or `api.js` — in Claude Code on the folder, or in the chat with the files pasted — **against rules you wrote into the prompt**, and you read it before you run it (cycle 5; cycle 2's you-do). Cowork, if you have it, may record your API's answer into `data/catalogue.json` — by hand it is the same two clicks.

NOT ALLOWED

Code you cannot explain line by line. A layer you did not break on purpose before you trusted it. **Any AI in the you-do of cycles 1, 3 and 4** — alone, like week 13.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Paste the files, copy back what returns, run the same review. Same grade.

CYCLE 1 · 25 MIN

BUILD THE LAYER: ONE FILE, ONE SUBSCRIBER, TWO GUARDS

The class library from week 10: correct, clean — and a reload throws it all away. You add `js/storage.js` under the key `library:v1`, wire it in two lines of `app.js`, and not one handler learns that anything is saved. Then the subscriber that reports into state — and loops forever.

1 I DO

10 MIN

3 on the next five slides, 7 in the editor · live-code/01-build-the-layer · `storage.js` · `hydrate(load())` + `subscribe(persist)` · the sentence into state (wrong: the first click never ends) · only when it changed · the identity guard

2 WE DO

5 MIN

you type steps 1, 2, 4, 5 · first visit is EMPTY · samples, reload, still there · compare with checkpoint 1

3 YOU DO

8 MIN

rename a book on `book.html` — an update by id, and it survives a reload with no save line

4 DEBRIEF

2 MIN

a subscriber that writes to state must know when not to · ↗ reference

YOU DO · AI-FREE

▸ REFERENCE · 3–8, 10–13, 15

EVERYTHING YOU BUILT LAST WEEK IS GONE

WHAT WE HAVE

One state object, one render function, handlers that only change state. Correct, and every bug from week 9 is impossible.

WHAT A USER DOES

Adds nine items. Closes the tab.
Comes back tomorrow.

WHAT THEY FIND

The six sample books, and none of theirs. **There is no error and no warning** — the application is working exactly as written.

Modern web apps are data-driven: data comes from a server, data is stored temporarily or permanently in the browser, and data is exchanged in a standard format.

THE WHOLE API, AND THERE IS NO SEVENTH LINE

```
1 localStorage.setItem(key, value); // store a key/value pair
2 localStorage.getItem(key);        // get the value by key
3 localStorage.removeItem(key);      // remove the key with its value
4 localStorage.clear();              // delete EVERYTHING for this origin
5 localStorage.key(index);           // the key at a given position
6 localStorage.length;              // how many are stored
```

► `clear()` is the one to be frightened of. It empties the **ORIGIN**, not your page.

ONE ORIGIN, AND EVERYTHING YOU HAVE EVER BUILT IS IN IT

every key this origin already holds · and 5KB into a cookie

↶ reload

open ↷

THE SAME STRING IN THREE PLACES, ON ONE ORIGIN

origin: http://127.0.0.1:56046 · path: /weeks/week-11-data-layer/demos/17-where-it-lives/index.html — the path is NOT part of the scope.

write "shalom" into all three

write 5KB into all three

read all three

remove what this demo wrote

	LOCALSTORAGE	SESSIONSTORAGE	COOKIE
what came back	null	null	null
size stored	—	—	—
survives a reload	yes	yes	until max-age
survives closing the browser	yes	NO	until max-age
shared with another tab	yes	NO — per tab	yes
sent to the server	no	no	no
room	~5MB per origin	~5MB per origin	~4KB, and refused silently

EVERY KEY THIS ORIGIN IS ALREADY HOLDING

(empty)

localStorage is scoped to the ORIGIN — scheme, host and port — and not to the folder. Every project you have ever opened with Live Server on this machine is in that list. That is the whole argument for a namespaced key.

sessionStorage is per TAB, and no page can show you that from inside itself. Open this file in a second tab, press "read" in both, and compare: localStorage agrees across the two, sessionStorage does not. Duplicating a tab copies it once and then the two diverge.

▶ localStorage is scoped to **SCHEME + HOST + PORT**, not to the folder. Every Live Server project on this machine shares one drawer.

WHERE DOES `save()` GO?

WHAT EVERYBODY WRITES FIRST

```
function onAdd() { ...; save(); }  
function onEdit() { ...; save(); }  
function onRemove() { ...; save(); }  
function onBump() { ...; save(); }
```

It works — and a **fifth** handler is a fifth chance to forget, invisible until a reload.

ONE LINE, ONCE

```
subscribe(render);  
subscribe(persist);
```

Not one handler learns that persistence exists. **They cannot forget, because they were never told.**

The same argument as week 10's handlers that draw, one layer further out.

THE WHOLE STORAGE HALF, AND IT IS TWO LINES OF `app.js`

```
1 hydrate(load());      // ← disk first, BEFORE anybody listens
2
3 subscribe(render);
4 subscribe(persist);   // ← and the disk follows every change
5 wire();
6 render(getState());
```

`load()`

Always an array. **No key is an empty library** — not the samples.

`render.js` · `events.js`

Not opened for storage. Not one handler calls `save`.

NOT `load()` ?? SEED

A boot that seeds when the read is empty cannot tell “nothing saved” from “the read failed”.

THE SUBSCRIBER THAT WRITES TO STATE HAS TO KNOW WHEN NOT TO

STEP 3 – “A FAILED WRITE IS SAID”

```
export function persist(state) {  
  const result = save(state.books);  
  setState({ storageNote: result.reason });  
}
```

STEPS 4-5 – TWO GUARDS

```
let lastSaved = null;  
  
export function persist(state) {  
  if (state.books === lastSaved) return;  
  const result = save(state.books);  
  if (result.ok) lastSaved = state.books;  
  if (result.reason !== state.storageNote)  
    setState({ storageNote: result.reason });  
}
```

First click: Maximum call stack size exceeded,
4,000+ writes.

Only a **changed** sentence is news. Typing is no
longer a write.

```
let n = 0; const set = Storage.prototype.setItem;  
Storage.prototype.setItem = function (...a) { n += 1; return set.apply(this, a); };
```

CYCLE 1 · YOU DO · AI-FREE · 8 MIN

RENAME A BOOK ON ITS OWN PAGE — AND WRITE NO SAVE LINE

THE TASK

- In `book.html`, **outside** `#book` (`render` replaces it on every change): a form `#rename` with a field `#new-title` and a button.
- In `events.js`: on `submit`, read the book's id from the address, and replace **that** book with a renamed copy — one `setState`, the way `+1` does it.
- No `save()`, no `localStorage` in `events.js`. `persist` is already listening.

CHECK YOURSELF — YES OR NO

1. Rename שערון המים to שערון החול and reload `book.html?id=b3`: does the page show the new name?
2. Back on the list: is the new name there, and is the **number of rows unchanged**?
3. Your `events.js` has no `save()` and no `localStorage` — and the name survived the reload anyway?

► The U of CRUD — the project grades it on the item page. The code goes on screen in the debrief, not before.

CYCLE 1 · DEBRIEF

THE RENAME, AS IT READS — AND NOT ONE SAVE LINE

```
1  const rename = document.querySelector('#rename');
2  rename?.addEventListener('submit', (event) => {
3    event.preventDefault();
4    const id = new URLSearchParams(location.search).get('id');
5    const title = document.querySelector('#new-title').value.trim();
6    if (title.length < 2) return;
7    setState({
8      books: getState().books.map((book) => (book.id === id ? { ...book, title } : book)),
9    });
10   rename.reset();
11 });
```

► #rename sits **OUTSIDE** #book, because render replaces #book on every change. Reload: the new name is there. persist did it.

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS

The screenshot shows a web application for library management. At the top, there is a navigation bar with links for 'ספר' (Book), 'הרשימה' (List), and 'ספריית ההשאלה' (Library). The main heading is 'ספריית ההשאלה של הכיתה' (Classroom Lending Library). Below this, there are two main sections. The first section is for adding a new book, with fields for 'שם הספר' (Book Name), 'שנת הוצאה' (Year of Publication), and 'מחיר' (Price), and a green 'הוסף' (Add) button. The second section is for managing the library, with fields for 'חידוש' (Renewal), 'מחיר' (Price), 'סדר' (Order), and 'לפי שם' (By Name), and a 'הוסף' (Add) button. Below these sections, there is a message: 'הספרייה ריקה. הוסף ספר בטופס, או טען ספרים לדוגמה.' (The library is empty. Add a book in the form, or load example books.) and a button 'טען ספרים לדוגמה' (Load example books).

Compare — a first visit is an **empty** library with the offer to load samples. Load them, press +1, reload: six rows, the new count. In Application: `library:v1`, and its value starts with `[]`.

CYCLE 2 · 25 MIN

CORRUPT IT, AND SURVIVE: PARSE, THEN CHECK

Edit the saved value to `{{{` and reload: nothing, and a `SyntaxError`.
`try` around the parse survives that — and dies on
`{"not":"an array"}`. Then `load` the project's way: never throws,
always an array, a sentence into state.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/02-corrupt-and-survive · try around the parse only (wrong) · access, parse, `Array.isArray`, filter, the sentence · the week-13 fault, typed and undone

2 WE DO

5 MIN

you type steps 1–2, then four values in Application: `{{{` · `null` · `{"not":"an array"}` · a bad row

3 YOU DO

8 MIN

a failed write, said in a sentence the user can act on — `QuotaExceededError`, checked against MDN

4 DEBRIEF

2 MIN

try/catch is a quarter of the answer · ↗ reference

YOU DO · AI AS PARTNER

➤ REFERENCE · 16–29

THE ACCESS THROWS, NOT JUST THE PARSE

WHAT EVERYBODY WRITES

```
const raw = localStorage.getItem(KEY);
try {
  return JSON.parse(raw);
} catch {
  return null;
}
```

Line 1 is outside the `try`, and line 1 is the one that throws in a browser with storage blocked.

THE LAYER

```
let text;
try {
  text = localStorage.getItem(KEY);
} catch {
  // draw anyway
  return { books: [], reason: 'blocked' };
}
```

Private mode, blocked cookies, partitioned storage — all the same throw.

A throw on the boot path is a blank page.

ABSENCE IS NOT CORRUPTION — AND IT IS NOT THE DEMO DATA EITHER

WHAT HAPPENED	LOAD() RETURNS	AND WHY
<code>getItem</code> threw	<code>[]</code> + a sentence	storage is blocked. Not a reason to fail to draw
<code>getItem</code> returned <code>null</code>	<code>[]</code> , no sentence	a first visit: an EMPTY library
<code>JSON.parse</code> threw	<code>[]</code> + a sentence	corrupt — and leave the string alone
parsed, not an array	<code>[]</code> + a sentence	<code>null</code> and <code>{"not": "an array"}</code> are valid JSON
an array with one bad row	the good rows + a sentence	one old row costs one row, not forty

THE STORAGE CONTRACT, IN FOUR LINES

1 The key is `<app>:v1` — `library:v1`, `pantry:v1`. Not `books`, not `ruppin.pantry.v1`.

The value is

2 **the array itself**

: `JSON.stringify(items)`. No envelope around it.

3 **One**

`getItem` in `js/storage.js`, and it reads the collection.

4 **No key is an empty collection.**

The samples come from a button, never from an empty read.

WEEK 13, VERSION B PLANTS THIS

```
text = sessionStorage.getItem(KEY);
```

AND YOU SEE

An empty library — while Application still holds all six. **With a seeding boot you would see six samples and never know.**

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS

The screenshot shows a web application for library management. At the top, there is a navigation bar with links: "ספר" (Book), "הרשימה" (List), and "ספריית ההשאלה" (Library). Below this, the main heading is "ספריית ההשאלה של הכיתה" (Classroom Lending Library). The interface is divided into three main sections:

- Top Section:** Contains three input fields labeled "שם הספר" (Book Name), "עותקים" (Copies), and "מדור" (Category). A green button labeled "הוסף" (Add) is positioned to the left of the "שם הספר" field.
- Middle Section:** Contains a "חיפוש" (Search) input field, a "מדור" (Category) dropdown menu with "הכול" (All) selected, and a "סדר" (Order) dropdown menu with "לפי שם" (By Name) selected.
- Bottom Section:** Displays the text "מוצגים: 0" (Items: 0) and a message: "הספרייה ריקה. הוסף ספר בטופס, או טען ספרים לדוגמה." (The library is empty. Add a book in the form, or load example books). Below the message is a button labeled "טען ספרים לדוגמה" (Load example books).

Compare — with the key deleted: the empty library. Then four values in Application, a reload after each: `{{}}` and `null` and `{"not": "an array"}` come up empty **with a sentence**; a book beside `{"junk": true}` comes up as **one row**. The console stays clean.

CYCLE 3 · 25 MIN

FETCH, THEN BREAK IT: `res.ok`, A TIMEOUT, THE OFFLINE PATH

A fifth layer, `js/api.js`, asks Open Library. The four lines everybody writes find the hobbit — and turn a **422** into “no such book”. Then every answer is checked, every failure has a sentence, and the local catalogue — a recorded answer — goes through the same normaliser.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/03-fetch-then-break · fetch + json (wrong: “ab” is a lie) · `res.ok` · timeout and a second try · the local data/catalogue.json · file://

2 WE DO

5 MIN

you type steps 1–2 and 4 · everyone types “ab” before the fix, with Network open

3 YOU DO

8 MIN

your project's API: the four questions, checked in the browser · one request CORS blocks

4 DEBRIEF

2 MIN

fetch does not throw on 422, 404 or 500 · ↗ reference

YOU DO · AI-FREE

➤ REFERENCE · 30–32, 34–39

FETCH RESPONSE IS NOT THE DATA — THIS IS A CRITICAL CONCEPT

six responses through two loaders↻ reloadopen ↗

SIX RESPONSES, TWO LOADERS

1 · 200, good JSON 2 · 404, valid JSON 3 · 500, valid JSON 4 · 200, an HTML login page 5 · 200, empty body 6 · the network is gone

WHAT THE SERVER SENT

THE LOADER EVERYBODY WRITES

```
async function load(url) {
  const res = await fetch(url);
  const data = await res.json(); // no check of res.ok, ever
  return data.title;
}
```

—

THE LOADER IN THE ASSIGNMENT

```
async function load(url) {
  const res = await fetch(url); // rejects ONLY if it never completed
  if (!res.ok) throw new Error('HTTP ' + res.status); // <- the whole slide
  let data;
  try { data = await res.json(); } // a SECOND thing that can fail
  catch { throw new Error('the body was not JSON'); }
  if (typeof data?.title !== 'string') throw new Error('not the shape we expect');
  return data.title;
}
```

—

CASE	PROMISE REJECTED?	RES.OK	RES.STATUS	NAIVE LOADER
do it for real, over the network				

- Run case 1, then case 2, and ask what `try/catch` would have done. Write the answer on the board **BEFORE** running case 6.

COMMON MISTAKES

- 1 forgetting `return response.json()`
- 2 forgetting `await`
- 3 assuming `fetch` throws on 404 — **it doesn't**

A `fetch()` promise only rejects on network errors. It does not reject on HTTP error statuses like 404 or 500.

It rejects for one reason: the request never completed. DNS, no route, CORS, abort.

STATUS CODES, IN FOUR FAMILIES AND ONE QUESTION

2XX — IT WORKED

200, 201 Created, **204 No Content**. `res.ok` is true for all three — which is why `status === 200` is the wrong test.

3XX — LOOK ELSEWHERE

`fetch` follows these for you by default. You usually never see one.

4XX — YOU GOT IT WRONG

400 bad request, 401 not logged in, 403 not allowed, 404 no such thing, **422 cannot process that** (Open Library, under three letters), **429 too many**.

5XX — THEY GOT IT WRONG

500, 502, 503. Nothing you send will fix it, and retrying hard makes it worse.

Your code asks one question: `res.ok`.
The number is for the message.

THE OFFLINE PATH IS A RECORDED ANSWER, THROUGH THE SAME NORMALISER

API.JS – STEP 4

```
const CATALOGUE = 'data/catalogue.json'; // the PAGE's path

export async function searchOffline(term) {
  const res = await fetch(CATALOGUE);
  if (!res.ok) throw new CatalogueError('status', '...');
  const body = await res.json();
  const needle = term.trim().toLowerCase();
  return toSuggestions(body).filter((s) =>
    s.title.toLowerCase().includes(needle));
}
```

WHY A RECORDING, AND WHY HTTP

data/catalogue.json is what search.json?q=library saved as it came. Two sources through **one** toSuggestions apart.

Offline, click index.html instead: says what failed **and** offline, click script at 'file:///.../js/app.js' from Live Server, always.

CYCLE 3 · YOU DO · AI-FREE

BEFORE YOU BUILD ON AN API: FOUR QUESTIONS, IN THE BROWSER

NO KEY?

A key in client-side JavaScript is a key you published.

OPEN TO CORS?

`access-control-allow-origin` on the answer, in the Network

KNOWS YOUR ENTITY?

A book catalogue will not help a pantry.

ONE REQUEST?

Two per search is twice the budget for the same answer.

```
await fetch('https://he.wikipedia.org/w/api.php?action=query&list=search&srsearch=test&format=json');  
// TypeError: Failed to fetch - "blocked by CORS policy". Add &origin=* and it works.
```

CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS

The screenshot shows a web application for a library. At the top, there is a navigation bar with the text "ספריית ההשאלה" (Library) and "הרשימה" (List). Below this, the main heading is "ספריית ההשאלה של הכיתה" (Classroom Library). The form contains several input fields and buttons:

- A search bar with a green "הוסף" (Add) button.
- Fields for "שם הספר" (Book Name), "עיתקים" (Copies), and "מדור" (Category).
- A "חיפוש" (Search) button.
- Fields for "סדר" (Order) and "מדרג" (Rank).
- A "לפי שם" (By Name) dropdown menu.
- A "הכול" (All) dropdown menu.
- A "מובנים: 0" (Built: 0) label.
- A "הספרייה ריקה. הוסף ספר בטופס, או טען ספרים לדוגמה." (The library is empty. Add a book in the form, or load example books.) message.
- A "טען ספרים לדוגמה" (Load example books) button.
- A section titled "הוספה מהקטלוג של Open Library" (Add from Open Library Catalog).
- A "חיפוש ספר בקטלוג" (Search book in catalog) input field.
- A "הקלד שם של ספר כדי לחפש בקטלוג." (Enter the name of a book to search in the catalog.) label.

Compare — then with Network open: **hobbit** gives eight suggestions; **ab** gives the error region with (422) and the offer; Offline and **library**, then the offer: ten suggestions from the recorded copy.

BREAK

10 MINUTES

Back at 95. Cycle 4 is the five states and the catalogue into the library; cycle 5, at 120, is the AI cycle — a chat window open, free or paid, and `ai-lab/05-save-and-load/artifact/start/` in your editor.

CYCLE 4 · 25 MIN

FIVE STATES, ONE FIELD — AND THE CATALOGUE INTO THE LIBRARY

A `loading` boolean puts “searching...” and “no such book” on screen together. Then one field, a derived view in `data-state` — and a five-line handler that adds a suggestion to the library, on disk after a reload.

1 I DO

10 MIN

3 on the next four slides, 7 in the editor · live-code/04-five-states-one-field · a loading boolean (wrong) · status + selectCatalogueView + data-state · button.adopt, one listener, derived suggestions

2 WE DO

5 MIN

you type steps 2–3 · watch data-state in Elements · adopt, reload, still there

3 YOU DO

8 MIN

the search box: a 300 ms debounce, one AbortController, and an abort that is not an error

4 DEBRIEF

2 MIN

two booleans are combinations nobody wrote · ↗ reference

YOU DO · AI-FREE

➤ REFERENCE · 40–51

WEEK 7 NAMED FOUR STATES. TWO OF THEM START TODAY.

WEEKS 7 – 10

empty and **success**. Reading an array from memory is instant, and reading it from disk (cycle 1) cannot fail halfway.

TODAY

loading and **error** finally have something to do — because a request takes time and can fail.

AND A FIFTH

idle. “Nobody has asked” is not “I asked and there was nothing”.

FIVE STATES, OR THIRTY-TWO COMBINATIONS

the same journey through four booleans and through one field

↶ reload ↷ open ↶

FIVE STATES, TWO WAYS TO KEEP THEM

idle loading **results** empty error the sequence a real user produces reset

FOUR BOOLEANS

```
let isLoading = false;
let isError  = false;
let isEmpty  = false;
let results  = [];
```

פפריקה מעושנת

פלפל שחור

כמון טחון

1 region visible — which is the only number that ever means anything

ONE FIELD, FIVE VALUES

```
state.search = { status, results, error };

view = status === 'loading' ? 'loading'
      : status === 'error'  ? 'error'
      : status === 'idle'   ? 'idle'
      : results.length === 0 ? 'empty'
      : 'results';
```

פפריקה מעושנת

פלפל שחור

כמון טחון

1 region visible — which is the only number that ever means anything

STEP

WHAT HAPPENED

FOUR BOOLEANS: REGIONS VISIBLE

ONE FIELD: REGIONS VISIBLE

► Press **THE SEQUENCE A REAL USER PRODUCES**. The left panel contradicts itself three times. **NOBODY WROTE THAT COMBINATION.**

FIVE STATES, ONE ATTRIBUTE, AND ONE DERIVATION

```
1 export function selectSearchView(state) {  
2   const { status, results } = state.search;  
3   if (status === 'loading') return 'loading';  
4   if (status === 'error') return 'error';  
5   if (status === 'idle') return 'idle';  
6   return results.length === 0 ? 'empty' : 'results'; // DERIVED  
7 }  
8  
9 // render.js - one value, four lines that all read it  
10 const view = selectSearchView(state);  
11 panel.dataset.state = view;  
12 document.querySelector('#api-loading').hidden = view !== 'loading';
```

► empty is **DERIVED**. loading is **STORED**. Nothing on the screen tells you whether a request is open.

FOUR FAILURES, AND WHAT EACH ONE LOOKS LIKE TO A PERSON

WHAT HAPPENED	WHAT YOUR CODE SEES	WHAT THE USER MUST BE TOLD
bad URL → 404	a resolved response, <code>res.ok === false</code>	“the catalogue returned an error (404). It is not something you typed.”
their server broke → 500	the same, with a different number	the same sentence — and no retry button , because it will not help
slow network	nothing, for as long as it takes	a loading state , then “this is taking too long” and a way to try again
offline	<code>TypeError: Failed to fetch</code>	“check your connection” — and the offer of the local catalogue

► Two of these look **IDENTICAL** to a user with no error state: nothing happens, twice, for opposite reasons.

FROM THE CATALOGUE INTO THE LIBRARY — FIVE LINES, AND NO SAVE

```
1  const suggestions = document.querySelector('#cat-results');
2  suggestions?.addEventListener('click', (event) => {
3    const row = event.target.closest('li[data-title]');
4    if (!row || !event.target.closest('button.adopt')) return;
5    const { books, search } = getState();
6    const suggestion = search.results.find((s) => s.title === row.dataset.title);
7    if (suggestion === undefined) return;
8    setState({ books: [...books,
9      { id: newId(), title: suggestion.title, copies: 1, genre: 'from the catalogue' }] });
10 });
```

► One listener, on a list `render` empties but never replaces. The row says **WHO**; the data comes from **STATE**. And `selectNewSuggestions` derives what is still worth offering — remove the book, and its suggestion comes back.

THE SEARCH BOX THAT ANSWERS THE WRONG QUESTION

two boxes, the same server – and a counter for disagreements

↶ reload

open ↷

TYPE THE SAME WORD IN BOTH BOXES

fix on the right:

debounce + abort + token

abort only

token only

reset counters

NO DEBOUNCE, NO ABORT, NO TOKEN

search

requests sent: 0 · answers applied: 0 · disagreements: 0

—

DEBOUNCE + ABORT + TOKEN

search

requests sent: 0 · answers applied: 0 · disagreements: 0

—

THE FOUR LINES

```
let controller = null;           // 1 – one per box
let seq = 0;                     // 2 – a ticket per search

async function run(query) {
  controller?.abort();           // 3 – the previous one is not wanted
  controller = new AbortController();
  const ticket = ++seq;

  const results = await search(query, { signal: controller.signal });

  if (ticket !== seq) return;     // 4 – a newer search has started
  show(results);
}
```

► Type **PAPRIKA** slowly in the left box and say nothing. Let somebody notice. This is cycle 4's you-do.

TWO REASONS TO STOP, ONE SIGNAL

WHAT AN ASSISTANT WRITES

```
1 const c = new AbortController();
2 const t = setTimeout(() => c.abort(), 4000);
3 const res = await fetch(url, { signal: c.signal });
4 // ...and the clearTimeout in a finally
5 // which is the line that gets left out
```

WHAT IT IS

```
1 const signal = AbortSignal.any([
2   caller.signal,           // AbortError
3   AbortSignal.timeout(4000), // TimeoutError
4 ]);
5 const res = await fetch(url, { signal });
```

- any forwards whichever fired first **WITH ITS REASON** — which is exactly what the `catch` needs.

CYCLE 4 · YOU DO · AI-FREE · 8 MIN

THE SEARCH BOX THAT ANSWERS THE WRONG QUESTION: WAIT, THEN CANCEL

THE TASK — IN `EVENTS.JS` AND `API.JS`

1. **Wait 300 ms** after the last keystroke — `clearTimeout`, then `setTimeout`. An empty box is `idle` at once.
2. **One `AbortController`**: a new search aborts the previous one;
`searchCatalogue(term, { signal })` passes it on, with the timeout.
3. **An abort is not an error**: `AbortError` passes through `api.js`, and the handler returns on it.

CHECK YOURSELF — YES OR NO

1. **hobbit** typed without a pause: **one** request in Network, not six?
2. Slow 4G, **hob**, half a second, **bit**: is the first request **(canceled)**?
3. Did the panel **never** reach `data-state="error"` ?

► Part F of the assignment. The whole handler comes in the debrief.

FOUR LINES, AND EACH ONE STOPS SOMETHING ELSE

```
1 let controller = null;
2 let seq = 0;
3 let timer; // the debounce
4 async function run(query) {
5   controller?.abort(); // 1 - stops the NETWORK
6   controller = new AbortController();
7   const ticket = ++seq; // 2 - a ticket
8   patch({ status: 'loading', error: null }); // 3 - BEFORE the await
9   try {
10    const results = await search(query, { signal: controller.signal });
11    if (ticket !== seq) return; // 4 - stops the WRITE
12    patch({ status: 'done', results });
13  } catch (error) {
14    if (error.name === 'AbortError') return; // an abort is not an error
15    patch({ status: 'error', error: sentence(error) });
16  }
17 }
18 // in the input handler: wait 300 ms after the last keystroke
19 clearTimeout(timer);
20 timer = setTimeout(() => run(query), 300);
```

► A search the next one aborted: **(CANCELED)** in the Network tab, `net::ERR_ABORTED` to the browser's own tools — and **NO** red panel.

“IF IT ABORTS, WHY DO I NEED THE NUMBER?”

`abort()` IS SYNCHRONOUS

It stops the request. Immediately, and for good.

THE `await` IS NOT

A response that had **already arrived and resolved** when the abort landed still runs its continuation.

AND SOME THINGS CANNOT BE CANCELLED

A third-party SDK. A library that takes no signal. The ticket is the **only** fix there.

The abort stops the work. The ticket decides whether the answer is still wanted.

CHECKPOINT 4

YOUR SCREEN SHOULD LOOK LIKE THIS

ספריית ההשאלה [הרשימה](#) ספר

ספריית ההשאלה של הכיתה

שם הספר: עותקים: מדור:

חידוש: מדור: סדר:

מוצגים: 0

הספרייה ריקה. הוסף ספר בטופס, או טען ספרים לדוגמה.

הוספה מהקטלוג של Open Library

חידוש ספר בקטלוג:

הקלד שם של ספר כדי לחפש בקטלוג.

Compare — then in Elements, `<section id="cat-panel" data-state="idle">` becomes `loading`, then `results`, one region at a time. Load the samples, add **The Hobbit**, reload: it is there. And homework 4 uses the same field with other words — `<li data-state="fresh">`.

CYCLE 5 · 25 MIN

“SAVE IT, AND SEARCH OPEN LIBRARY” — REVIEW IT AGAINST TODAY’S THREE RULES

Prompt 1 is two wishes and three files. What returns is short, confident, and breaks every rule you typed today: `persist` and `setState` call each other, `JSON.parse` has no `try`, and `fetch` never looks at `res.ok`. You find all three in the browser before anyone names them.

1 I DO

7 MIN

3 on the next two slides, 4 in the chat · ai-lab/05-save-and-load · P1 live · paste it · the good journal entry on screen

2 WE DO

6 MIN

console on load · `{{{` in its key · "ab" with Network open

3 YOU DO

10 MIN

your own P2 with the rules written in · review before you run · one journal line

4 DEBRIEF

2 MIN

the 422 is the one nobody catches · ↗ reference

YOU DO · AI AS PARTNER

PROMPT 1, AND WHAT CAME BACK

THREE FAULTS IN A DATA LAYER THE MACHINE WROTE

THE PROMPT, WORD FOR WORD (ENGLISH)

I have a class-library app. Two things:

1. Save the books in localStorage, so they are still there after a reload.
2. Make the search box with id="cat-q" search Open Library as the user types, and show the results in #cat-results.

Here are state.js, events.js and app.js.

THREE EXCERPTS OF WHAT CAME BACK

```
export function setState(patch) {  
  state = { ...state, ...patch };  
  persist(state); // save on every change, so nothing is ever lost  
  ...  
export function persist(state) {  
  localStorage.setItem(KEY, JSON.stringify(state.books));  
  setState({ savedAt: new Date().toISOString() });  
}  
export function load() {  
  return JSON.parse(localStorage.getItem(KEY));  
}  
const res = await fetch(`...search.json?q=${query}&limit=8`);  
const data = await res.json();
```

CYCLE 5 · ON SCREEN BEFORE ANYONE WRITES

WHAT A JOURNAL ENTRY THAT IS WORTH SOMETHING LOOKS LIKE

WHAT I ASKED · WHAT CAME BACK

The prompt, word for word. What returned, in one line — including what you did not ask for.

WHAT WAS WRONG, AND HOW I SAW IT

A thing you did in the browser — “the console said Maximum call stack size exceeded on load”, “ab returned 422 and the page said no books”. Then what you kept, and what you learned about the prompt.

► **AI-EXEMPLARS/EXEMPLARS/PROMPT-JOURNAL-GOOD.MD** in your course materials — and **PROMPT-JOURNAL-BAD.MD** beside it. “It didn’t work” is not a line.

CHECKPOINT 5

THREE QUESTIONS, THREE YESES

1. Did you find **all three** faults in the answer to P1 — each one **in the browser** (console · Application · Network)?
2. Is every rule written in your P2 as a condition you can check in the answer — and did you check it **by reading, before running**?
3. Does your journal line say **what you saw** — a message, a status, a number?

CLASS ASSIGNMENT 11: THE SAME TWO LAYERS, ON THE PANTRY

145–175 · IN CLASS

Part A: `js/storage.js` under `pantry:v1` — the layer you typed in cycles 1–2. Commit when it survives a reload.

AT HOME, BEFORE LESSON 12

Edit on the list page · `js/api.js` · the five states · the catalogue into the pantry. Then the stretch: the race and the write budget.

BEFORE YOU SUBMIT

Delete the key and reload: an **empty** pantry. Wi-Fi off and search: a sentence and the local copy. `js/app.js` : not edited.

► Then copy `js/storage.js` and `js/api.js` into your project — they are written to its contract. Next week: `api.js` becomes TypeScript, and a wrong guess about somebody else's answer is caught **BEFORE** you run.

CLASS ASSIGNMENT 11 · PART 6 · THE TWO FORMS NO CYCLE TYPED

ONE RETRY, AND THE ONE “ARE YOU SURE?”

API.JS – ASK AGAIN ONLY WHEN NO ANSWER ARRIVED

```
export async function searchAgain(term, signal,
                                  retries = 1) {
  for (let attempt = 0; ; attempt += 1) {
    try {
      return await searchCatalogue(term, { signal });
    } catch (error) {
      const retry = attempt < retries &&
        ['offline', 'timeout'].includes(error.kind);
      if (!retry) throw error; // a 422 twice is a 422
      await sleep(300 * 2 ** attempt, signal); // grows
    }
  }
}

const sleep = (ms, s) => new Promise((ok, fail) => {
  setTimeout(ok, ms); // the same signal ends the wait:
  s?.addEventListener('abort', () => fail(s.reason));
});
```

EVENTS.JS – CONFIRM ONLY WHAT CANNOT BE UNDONE

```
const clearAll = document.querySelector('#clear-all');
clearAll?.addEventListener('click', () => {
  // Cancel: nothing changes
  if (!confirm('להסיר את כל הספרים?')) return;
  // persist writes [] – no save line
  setState({ books: [] });
});
```

Cancel changes nothing; OK empties the list; a reload shows it empty — `persist` wrote `[]`.

► Written for the library, measured on cycle 4's `end/`: a dropped connection, then an answer — **2 REQUESTS, RESULTS**; a 422 — **1 REQUEST, THE ERROR SENTENCE**. On the pantry, it is yours to assemble.

HOMWORK 4, OUT TODAY: A BOARD THAT KNOWS HOW OLD IT IS

THE SAME FIELD, OTHER WORDS

```
<li data-city="haifa" data-state="fresh">...</li>  
<!-- idle · loading · fresh · stale · error -->
```

Cycle 4's `data-state`, one per row. `fresh` and `stale` are derived from a saved `fetchAt` — a **number**, because a `Date` comes back from JSON as a string.

```
// from the future (clock set back)? fresh, age 0  
const ageMs = Math.max(0, now - entry.fetchAt);  
// "fresh" comes from the clock, so redraw once a minute  
setInterval(render, 60_000);
```

WHERE EACH PART WAS TYPED

- `cache.js` — cycles 1–2: a layer that never throws, parse then check
- `api.js` — cycle 3: `new URL`, `res.ok`, a timeout, `res.json()` in its own `try`
- five states per row — cycle 4; the abort — cycle 4's you-do
- `Promise.allSettled`, `migrate` — the next slide, now; demos 05 and 20 run them
- the challenge, stale-while-revalidate — the slide after that

HOMEWORK 4 · THE TWO FORMS NO CYCLE TYPED

SIX REQUESTS AT ONCE, AND A SHAPE FROM LAST YEAR

PROMISE.ALLSETTLED – NOT ALL, NOT SIX AWAIT'S IN A ROW

```
const results = await Promise.allSettled(
  tasks.map((task) => task()),
);
// never rejects – one verdict per task, in order:
//   { status: 'fulfilled', value }
//   { status: 'rejected', reason }
const failed = results.filter(
  (result) => result.status === 'rejected',
);
```

`Promise.all` rejects at the **first** failure, and the ones that landed are out of reach. Six `await` s in a loop take the **sum** of the six waits.

MIGRATE – THREE ANSWERS, NEVER A THROW

```
function migrate(parsed) {
  // today's shape: take it
  if (parsed?.v === 1 && Array.isArray(parsed.items)) {
    return parsed.items;
  }
  // version 0, before there was a v: adopt it
  if (Array.isArray(parsed)) return parsed;
  // a future version, or nonsense: start over
  return [];
}
```

Homework 4's cache keeps a versioned envelope; **your project does not** — its contract is a bare array under `<app>:v1`.

► Measured: five tasks with the third failing — `allSettled` gives four fulfilled and one rejected; `migrate` keeps v1 and v0, and returns `[]` for v2, null and nonsense. Demos 05 and 20 run both.

HOMEWORK 4 · THE CHALLENGE TIER, SHOWN NOT TYPED

SHOW WHAT YOU HAVE, THEN REFRESH IT QUIETLY

BOOT – THE ORDER IS THE PATTERN

```
render(); // 1. what you have, now – stale included
requestAnimationFrame(() => { // 2. after that paint
  for (const city of CITIES) {
    if (viewOf(city.id).view !== 'stale') continue;
    refreshCity(city.id, { quiet: true });
  }
});
```

INSIDE REFRESHCITY – THE { QUIET } IDEA

```
// 3. a row that already shows a number keeps it
const hasData = state.cache[cityId] !== undefined;
if (!(quiet && hasData)) setState({
  status: { ...state.status, [cityId]: 'loading' },
});
```

A background refresh that puts a spinner over a number has traded real data for a placeholder. A row with **nothing** still gets the spinner.

► This is called **STALE-WHILE-REVALIDATE**: the user reads the old number while the new one is on its way, and never waits for something you already have.

THREE QUESTIONS BEFORE YOU GO

- 1 You delete your app's key and reload.
What must be on the screen — and what would a boot that seeds on an empty read show?
- 2 The server answered **422** with a valid JSON body. Did your `catch` run?
- 3 `persist` calls `setState`.
Why does it not run forever?

GO DEEPER · 1 OF 2

The reference deck for this week — every slide the lecture pointed at, in order.

[3 · The same application twice. One of them writes it down.](#) ← spine 4

[4 · What the browser is actually willing to keep](#) ← spine 4

[5 · Two mistakes, both silent, both in the first hour](#) ← spine 4

[6 · Three places to put a string. One slide.](#) ← spine 4

[7 · How much room is there, actually](#) ← spine 4

[8 · What localStorage is not](#) ← spine 4

[10 · What a keystroke costs](#) ← spine 4

[11 · Two fixes, and both are right](#) ← spine 4

[12 · What belongs in state, and what belongs on disk](#) ← spine 4

[13 · The letter of CRUD that was missing](#) ← spine 4

[15 · The quota, measured — and why save returns a sentence](#) ← spine 4

[16 · JSON is a text format, and it has five rules](#) ← spine 14

[17 · The round trip, field by field](#) ← spine 14

[18 · Twelve values, three different answers each](#) ← spine 14

[19 · The Date, and the crash that arrives tomorrow](#) ← spine 14

[20 · The second argument, and what it costs](#) ← spine 14

[21 · Reading a nested structure you did not write](#) ← spine 14

[22 · An empty catch is worse than no catch](#) ← spine 14

[23 · Six things that are not your data](#) ← spine 14

[24 · Parse, then check — and check only what you read](#) ← spine 14

[25 · “Clear all” is saving an empty array](#) ← spine 14

[26 · The shape changed. Their browsers did not.](#) ← spine 14

[27 · Yesterday's data, three programs](#) ← spine 14

[28 · Three answers, not two](#) ← spine 14

GO DEEPER · 2 OF 2

The reference deck for this week, continued — in order.

[29 · What JSON.parse does with nothing](#) ← spine 14

[30 · The whole of fetch, and then the hard part](#) ← spine 19

[31 · The layer, and what is not in it](#) ← spine 19

[32 · Sending something: POST, and the one header that matters](#) ← spine 19

[34 · The most confusing error in front-end development](#) ← spine 19

[35 · The same code, two servers](#) ← spine 19

[36 · Why the browser does this at all](#) ← spine 19

[37 · There is no timeout, and there never has been](#) ← spine 19

[38 · CORS is a server's decision, and it changes](#) ← spine 19

[39 · 422 is an answer, not “nothing found”](#) ← spine 19

[40 · Everything on your screen was typed by you](#) ← spine 27

[41 · One thread, and what holding it costs](#) ← spine 27

[42 · The same job, three ways to write it](#) ← spine 27

[43 · A promise is three states, and it settles once](#) ← spine 27

[44 · try/catch works again, and that is the whole gift](#) ← spine 27

[45 · A rejection is not an exception until somebody asks](#) ← spine 27

[46 · async and await are the same thing, written flat](#) ← spine 27

[47 · What order does this actually run in?](#) ← spine 27

[48 · Five requests, and the loop that costs you five times](#) ← spine 27

[49 · all · allSettled · race, with a stopwatch](#) ← spine 27

[50 · Which one, and when](#) ← spine 27

[51 · Retry, and what it actually costs](#) ← spine 27

[53 · Two faults in a storage layer the machine wrote](#) ← spine 39

[54 · Three questions before you go — the storage half](#) ← spine 48